

Agenda

1) Pair Sum

2) Distinct elements in a subarray of size k .

Q1) Given N array elements, ^{boolean} check if there exists a pair (i, j) s.t. $arr[i] + arr[j] = k$ and $(i \neq j)$

arr[] = 8 9 1 -2 4 5 11 -6 7 5

i j arr[i] arr[j] → True
k=11 4 8 4 7

k=6 2 5 1 5 → True

k=22 6 6 11 11 → false
i+j

arr[i] arr[j]
↑ ↑
a+b == k or not

Idea 1:- Check all the pairs

```
for (i=0; i<n; i++) {
    int a = arr[i]; b = k-a;
```

```
    for (j=i+1; j<n; j++) {
        if (b == arr[j]) {
            return True;
        }
    }
}
```

return false;

T.C = $O(N^2)$

S.C = $O(1)$

→ can you optimise this part of code? → Yes

We need to only check if b is present in array or not → Hashset

Steps:-

1) Create a hashset → HashSet<int> hs;

2) Loop through arr[] and insert entire data in your hs.

3) In every loop, fix a, get value of $b = k - a$ and then check if b is present in hs.

arr[] =

0	1	2	3	4	5	6	7	8	9
8	9	1	-2	4	5	11	-6	7	5

hs:- $\langle 8, 9, 1, -2, 4, 5, 11, -6, 7 \rangle$

k=11

arr[i]	(k-a)
a	b
8	3 → No
9	2 → No
1	10 → No
-2	13 → No
4	7 → Yes (True)

k=6

arr[i]	(k-a)
a	b
8	-2 → True

k=22

arr[i]	(k-a)
a	b
8	14 → X
9	13 → X
1	21 → X
-2	24 → X
4	18 → X
5	17 → X
11	11 → ✓ (True)

→ You should be returning false here in reality.

→ If freq of 11 == 1 → Return false.

This approach is not working.

arr[] =

0	1	2	3	4	5	6	7	8	9
8	9	1	-2	4	5	11	-6	7	5

Approach 1:- Using Hash Map

1) Hash Map $\langle \text{int}, \text{int} \rangle$ hm

2) Iterate and insert in hm

hm:- { 3:1, -2:1, 11:1, 9:1, 4:1, -6:1, 1:1, 5:2, 7:1 }

Pseudo code:-

1) Create a hm & insert $\langle \text{arr}[i], \text{freq} \rangle$

2) for $(i=0; i < N; i++)$

```

    a = arr[i]; b = k-a;
    if (a != b) {
        if (b is in hm) {
            return True;
        }
    }
    else { // a == b
        if (hm[b] > 1) {
            return True;
        }
    }
}
return false;

```

arr[i]
a

8

9

1

2

4

5

11

(k-a)
b

k=22

14 → X

13 → X

21 → X

24 → X

18 → X

17 → X

11

if (a == b) { freq(a) > 1 }

Edge case.

T.C = $O(N)$

S.C = $O(1)$

→ Using Hashset

arr[] =

0	1	2	3	4	5	6	7	8	9
8	9	1	-2	4	5	11	-6	7	5

k=22

$a=4$ $a=5$ $a=11$ $a=-6$ $a=7$ $a=5$
 $b=18$ $b=17$ $b=11$ $b=28$ $b=15$ $b=17$

→ Return false

arr[] =

0	1	2	3	4	5	6	7	8	9	10
8	9	1	-2	4	5	11	-6	7	5	11

$a=4$ $a=5$ $a=11$ $a=-6$ $a=7$ $a=5$ $a=11$
 $b=18$ $b=17$ $b=11$ $b=28$ $b=15$ $b=17$ $b=11$

→ True

k=22

a	b	hs
8	14	{8}
9	13	{8, 9}
1	21	{8, 9, 1}
-2	24	{8, 9, 1, -2}
...		
11	11	{8, 9, 1, -2, 4, 5, 11}
...		
11	11	→ <u>Return True</u>

Pseudo Code :-

Hashset <int> hs;

for(int i=0; i<N; i++){

 a = arr[i];

 b = K - a;

 if (b is in hs) { return True }

 else { hs.insert(arr[i]) }

}

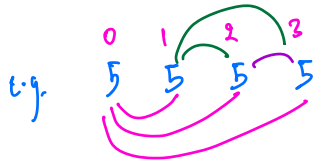
return false;

T.C = $O(N)$

S.C = $O(N)$

Single for loop solution.

Q2 calculate no. of (i, j) s.t. $arr[i] + arr[j] = K$ and $(i \neq j)$



$K=10$
total pairs = 6

hashtet $\rightarrow \{5\}$

hashtet <int> h; cnt=0

for(int i=0; i<N; i++){

$a = arr[i];$

$b = K - a;$

if (b is in h) { cnt++; }

else { decrement(arr[i]) }

}

return false;

a	b	cnt
5	5	0
5	5	1
5	5	2
5	5	3

X Hashtet won't work here
We need to use a hashmap.

Todo

Q3 Check if there exists a (i, j) s.t. $arr[i] - arr[j] = K$ ($i \neq j$)

$a = arr[i]$

$b = a - K$

————— X ————— X —————

Break:- 10:30

Strick No sorry policy.

Ques Given N array elements. Calculate no. of distinct elements in every window of size K .

arr:- $\overset{0}{2} \quad \overset{1}{4} \quad \overset{2}{3} \quad \overset{3}{8} \quad \overset{4}{3} \quad \overset{5}{9} \quad \overset{6}{4} \quad \overset{7}{9} \quad \overset{8}{4} \quad \overset{9}{10} \quad , K=4$

	# distinct
[0-3]	4
[1-4]	3
[2-5]	3
[3-6]	4
[4-7]	3
[5-8]	2
[6-9]	3

↓
print this data

Idea:-

for every window of size K , get the no. of distinct elements using a hashset.

N elements

K	# subarrays
1	N
2	$N-1$
3	$N-2$
$\vdots$	
$\vdots$	
K	$N-K+1$

Last subarray:- $[N-K, N-1]$
first subarray:- $[0, K-1]$

```
for (i=0; i <= N-K; i++) {
```

```
    HashSet<Int> hs;
```

```
    for (j=i; j < (i+K); j++) {
```

```
        hs.insert(arr[j]);
```

```
    }
```

```
    print(hs.size());
```

```
}
```

optimize this using sliding window.

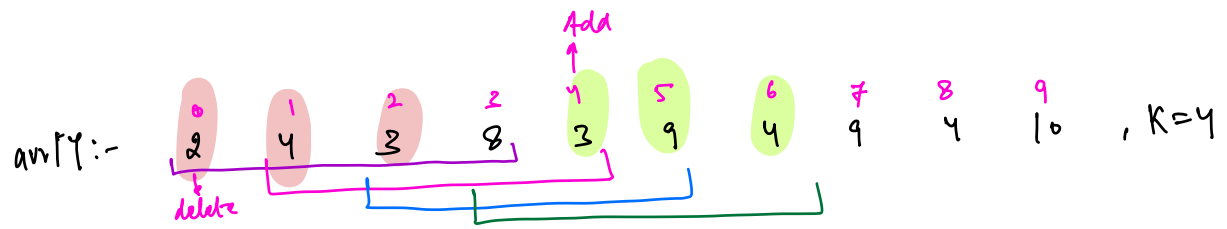
$[i, i+K-1] \Rightarrow \text{len} = K$

$$T.C = ((N-K+1) * K)$$

if $K = N/2$

$$T.C = O(N^2) \rightarrow W.C.$$

$$S.C = O(K)$$



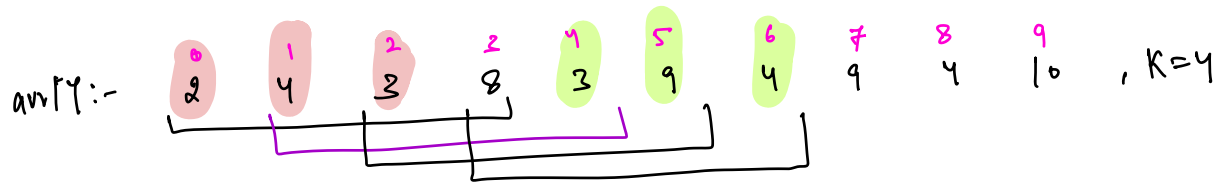
	remove	add	hashset	# count	# actual
[0-3]		All [0-3]	{2, 4, 3, 8}	4	4
[1-4]	2	3	{4, 3, 8}	3	3
[2-5]	4	9	{3, 8, 9}	3	3
[3-6]	3	4	{8, 9, 4}	3	4

↓
not working

We need to use a HashMap

// Hashmap with Sliding Window

hm.size()



	remove	add	hashmap <K,V>	# count	# actual
[0-3]		All [0-3]	{2:1, 4:1, 3:1, 8:1}	4	4
[1-4]	0th	4th	{2:0, 4:1, 3:2, 8:1} delete this	3	3
[2-5]	1st	5th	{4:0, 3:2, 8:1, 9:1}	3	3
[3-6]	2nd	6th	{3:1, 8:1, 9:1, 4:1}	4	4

In HM, if freq = 0 for a key, remove that key.

// Pseudo Code

```

HashMap <int, int> hm;
for (int i=0; i < K; i++) {
    // arr[i] insert in hm
    if (arr[i] is in hm) {
        hm[arr[i]]++;
    }
    else {
        hm.insert(arr[i], 1);
    }
}
print(hm.size());

```

1st Window:- [0, K-1]

2nd window:- [1, K]

3rd window:- [2, K+1]

4th window :- [3, K+2]

⋮

N-K+1th Window:- [N-K, N-1]

```
for (i=1, j=K; i<=(N-K+1); i++, j++) {
```

```
    // subarray [i, j]
```

// decrease freq. of (i-1)th element and increase freq/add (j)th element

```
    hm[arr[i-1]]--;
```

```
    if (hm[arr[i-1]] == 0) {
```

// remove element from hm

```
    }
```

```
    if (arr[j] is in hm) {
```

```
        // hm[arr[j]]++;
```

```
    }
```

```
    else {
```

```
        // hm.insert(arr[j], 1);
```

```
    }
```

```
    print(hm.size());
```

```
}
```

$T.C = O(N)$

$S.C = O(K)$

Q Find len of longest subarray with $\text{sum} = 0$ $\rightarrow$ Try this.

arr = 3 3 4 -5 -2 2 1 -3 3 -1 -5 -4 -1

pref = - - - - -

Next Class $\rightarrow$ Recursion 1 } $\rightarrow$ Monday
Recursion 2 } $\rightarrow$ Wednesday. $\rightarrow$ V. important

Backtracking

Dynamic Programming.

Heaps

Trees $\rightarrow$ 90% of questions using Recursion.

Tries